

Laboratory for Innovative Software (LIS)

Verification Framework Based on Hoare Logic and Incorrectness Logic

Individual Programming Project
Academic Year 2025-2026

Short technical report: language design, symbolic semantics, verification, API contracts,
examples, and limitations

1. Language syntax

Let x range over variables, n over integers, f over API names, and p over uninterpreted predicate names. The concrete syntax is described by this grammar:

```

e ::= n | x | -e | e + e | e - e | e * e | e / e | e mod e
    | load(e) | g(e1, ..., en)

b ::= true | false | e = e | e != e | e < e | e <= e | e > e | e >= e
    | !b | b && b | b || b | b => b | p(e1, ..., en)

c ::= skip | x := e | c ; c
    | if b then c else c
    | while b do c
    | store(e, e)
    | x := f(e1, ..., en)

api ::= f(x1, ..., xn) requires b ensures b
spec ::= {b} c {b}

```

Expressions have integer sort, assertions have Boolean sort, and memory addresses and stored values are integers. Division follows SMT-LIB integer division. API predicates and pure functions may be uninterpreted. The implementation is the typed OCaml AST in `lib/syntax.ml`; therefore malformed syntax cannot enter the executor.

2. Symbolic domain and memory

A symbolic state is $(\text{Env}, \text{Mem}, \text{PC}, \text{Path})$. Env maps source variables to symbolic integer terms. An absent variable denotes its own initial symbolic value. Mem is a functional SMT array from integer addresses to integer values. Initially it is the unconstrained array `mem0`; a write creates `store(Mem, address, value)`, while a read creates `select(Mem, address)`. PC is a list of accumulated assertions, and Path records branches, loop iterations, and API calls for diagnostics.

This functional memory representation automatically preserves aliasing. In particular, $Z3$ proves `select(store(M, a, v), a) = v` without a custom heap axiom.

3. Symbolic operational semantics

Write `eval(e, sigma)` for substitution of the current environment and memory into an expression, and `sat(F)` for an SMT satisfiability query.

- `skip` returns the input state.
- `x := e` updates `Env[x]` to `eval(e, sigma)`.
- `store(a, v)` updates memory to `store(Mem, eval(a), eval(v))`.
- `c1 ; c2` executes `c2` from every final state of `c1`.
- `if b then c1 else c2` forks into `PC` and `eval(b)` and `PC` and `not eval(b)`;

unsatisfiable forks are discarded.

- A loop forks into an exit path with `not b` and a body path with `b` at each

iteration. Exploration is bounded by the program's `loop_bound`.

- API calls use the contract rule described below.

The implementation's result also carries issues. If the loop guard remains feasible at the bound, verification is `Inconclusive` rather than claiming a proof. This distinction is important: finite unrolling alone is not a sound proof for unbounded executions.

4. Hoare-style verification

For $\{P\} \text{ c } \{Q\}$, execution begins with symbolic inputs, symbolic memory, and $PC = P$. For every feasible final state σ_i , the verifier asks Z3 whether

PC_i and not $\text{eval}(Q, \sigma_i)$

is satisfiable. If every query is unsatisfiable and there are no execution issues, the result is `Verified`. A satisfiable query is a failed proof and yields a concrete model. An unknown API or a feasible continuation beyond the loop bound yields `Inconclusive`. The property is partial correctness: termination itself is not proved.

5. Incorrectness-based bug finding

`Verifier.find_bug P command Q` (or the configurable `find_bug_program` variant) searches final paths for the under-approximate bug condition PC_i and not Q_i . A satisfiable result is a real execution of the symbolic semantics reaching a bad final state. The report contains:

- the failing path condition;
- the violated instantiated postcondition;
- Z3's concrete model for inputs and initial memory;
- the ordered branch/API/loop path.

API precondition violations are searched in the same way, using PC and not API_Pre . Thus failure to prove an API precondition is accompanied by a witness whenever one exists.

6. API specifications

An API contract stores a name, formal parameters, precondition, and postcondition. The reserved variable `result` in a postcondition denotes the returned value. For $x := f(a_1, \dots, a_n)$, the executor performs these steps:

- 1 evaluate actual arguments once in the caller state;
- 2 substitute them for the contract's formal parameters;
- 3 prove $PC \Rightarrow Pre$; if this fails, query PC and not Pre and report a witness;
- 4 continue only on the safe path by adding Pre ;
- 5 create a fresh symbolic result, instantiate $Post$, add it to PC , and bind x .

No implementation of the API is inspected. Contracts can mention arithmetic, pure uninterpreted functions, and predicates such as `allocated(result,n)` or `valid_key(k)`. They cannot currently describe memory mutation; extending a contract with an old/new memory relation would support stateful APIs.

7. Supplied programs

The five successful cases in `examples/programs.ml` cover branching absolute value, arithmetic assignment, store/load, a safe `safe_div` API call, and a bounded countdown. The five buggy cases cover the zero absolute-value corner case, an impossible arithmetic claim, an unjustified memory assumption, unsafe division with witness $b = 0$, and an incorrect stored value. Running the showcase prints each proof result and each model.

8. Design choices and limitations

The implementation emits SMT-LIB and invokes Z3 as a separate process. This avoids a platform-specific OCaml Z3 binding while retaining models and SMT arrays. Path feasibility is checked eagerly to control path explosion. Environments and memory are persistent, which makes forked states independent without copying mutable structures.

Current limitations are deliberate and visible in results:

- loops use bounded exploration; a feasible guard at the bound is inconclusive;
- the `invariant` AST field is reserved for a future inductive invariant rule but is not

used to claim unbounded proofs;

- arithmetic is mathematical integer arithmetic, not fixed-width machine arithmetic;
- division by zero must be ruled out by a program or API precondition;
- uninterpreted API predicates have no axioms unless encoded in the surrounding formula;
- there is no textual parser or source-location tracking; programs are typed OCaml ASTs;
- the executor is path-sensitive and may grow exponentially with nested branches.

These constraints preserve a small, auditable core while meeting the required language, memory, API, symbolic execution, Hoare verification, and optional bug-finding goals.